Meghan Gillen
May 1st, 2026

EE 446 - Lab 5

Questions:

1. In the notebook, the audio waveform is transformed into time-frequency features before being passed to the keyword spotting model. What is the purpose of converting raw audio into spectrogram/MFCC-style features instead of directly using the raw time-domain waveform?

The time domain waveform form of presenting information does not reveal as many patterns in the speech. By putting it into the frequency domain and revealing the harmonics, it is easier for the model to group information together. When looking at the different graphs that show the time domain audio vs the frequency domain, the volume of how a word is said has a huge impact on the similarity of the time domain graphs, but not the frequency domain. Since the model will look to classify the words based on their word not volume, that is important for its accuracy.

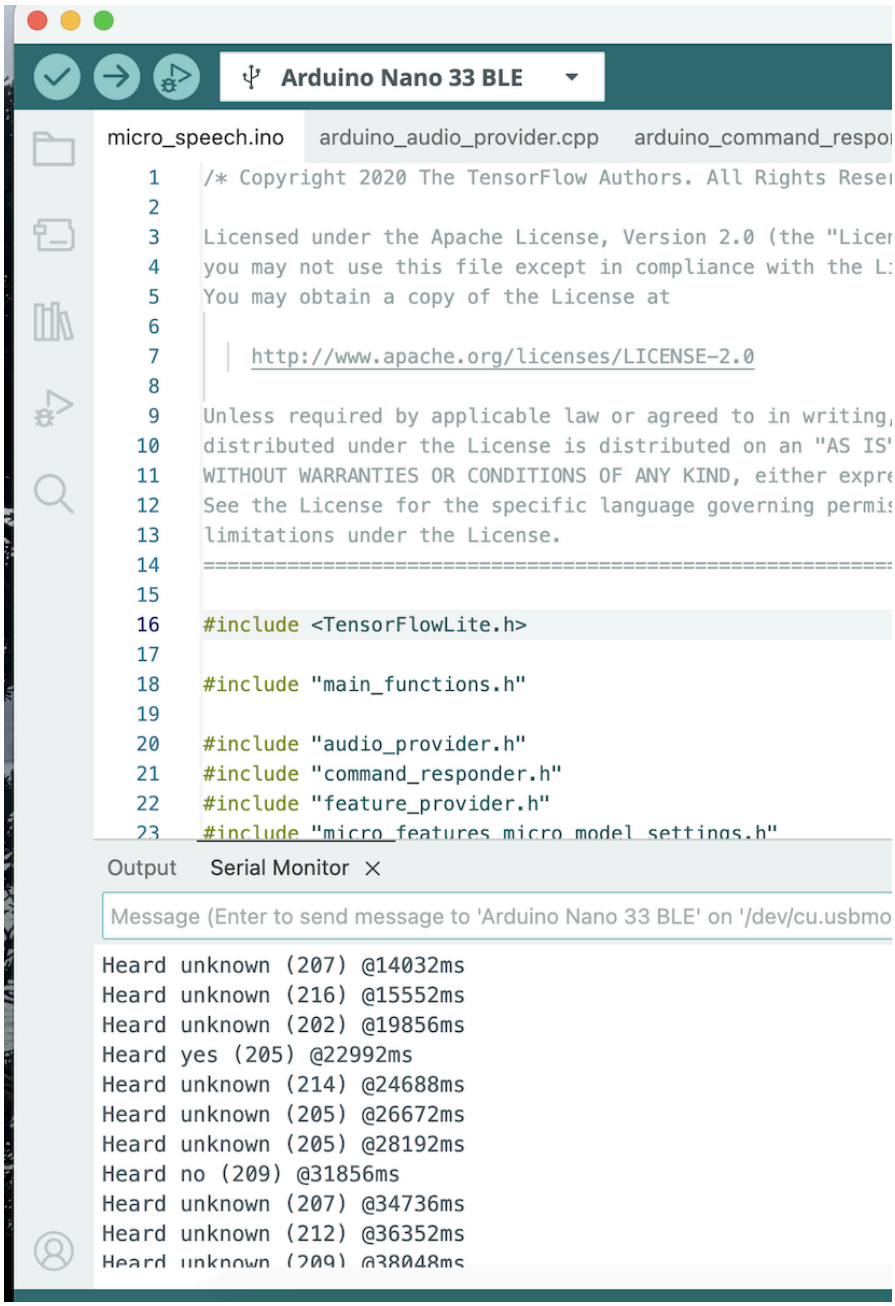
2. During full integer quantization, the notebook uses a representative dataset. What is the purpose of the representative dataset, and why is it important for creating an INT8 TensorFlow Lite model?

The purpose of the representative dataset is to find a realistic range of data, so the INT8 quantization will have a good guess of the range of data. By putting some real data through, it can find the range of values so when it is scaling the FP32 values to INT8 with a zero factor and scale, it is not the wrong range of values. That would decrease the accuracy of the INT8 model.

3. What does audio_processor.get_data() do? Refer to input_data.py.

This function loads and prepares a batch of data for training. It takes in the raw audio files from the disk, turns them into spectrograms and applies labels for the features. Therefore the result can be immediately put into the model.

4. In the final model export step, the TensorFlow Lite model is converted into a C/C++ byte array. What are g_model and g_model_len, and why do they need to be copied correctly into the Arduino source file?

G_model is the full binary of the model in hex values that make up the model, in a byte array. The g_model_len is the length of that array. The interpreter needs the hex values because it reads the array as a model in memory. If either is incorrect then the model is not correctly loaded.

5. Please include a screenshot showing the correct identification of at least one keyword, either “YES” or “NO”, as displayed in the Arduino IDE Serial Monitor after deploying your KWS model on the Arduino Nano 33 BLE. The screenshot should be included in your submitted PDF.